

07 — Build Roadmap & Acceptance Criteria

Build order for Antigravity. Do not generate the whole app blindly in one pass.

Phase 0 — Understand

- Read all seven project documents before coding.
- Inspect every image in /UI and summarize the common design language.
- Do not copy Pinterest assets or copyrighted UI exactly.
- Create a concise implementation plan and list assumptions.

Phase 1 — Foundation

- Create Flutter project.
- Set up architecture, theme, routing and reusable components.
- Implement onboarding and local persistence.
- Implement navigation shell.

Phase 2 — Cycle

- Implement period entry.
- Implement calendar.
- Implement cycle history.
- Implement deterministic calculations.
- Add unit tests before polishing screens.

Phase 3 — TTC

- Implement trying-to-conceive mode.
- Implement estimated fertile window.
- Implement optional fertility observations.
- Implement reminders.
- Add clear uncertainty and medical disclaimers.

Phase 4 — Pregnancy

- Implement positive-test flow.
- Implement pregnancy dating.
- Implement week/day display.
- Implement due-date estimate.
- Implement pregnancy dashboard, appointments and education.

Phase 5 — Privacy & Partner

- Implement app lock.

- Implement private fields.
- Implement explicit partner sharing model.
- Implement delete/export controls.

Phase 6 — AI

- Add AIService abstraction.
- Add provider adapter only after core app is stable.
- Use Gemini or OpenAI through a secure architecture.
- Add safety prompt and structured context.
- Add offline fallback.

Definition of Done

- App builds and runs without errors.
- No placeholder screens remain in the MVP path.
- All critical date calculations have automated tests.
- UI matches the design language from /UI while remaining original.
- App works with no network for core tracking.
- Sensitive data does not appear in logs.
- Every medical estimate is visibly labeled as an estimate.
- AI is optional and cannot break core functionality.

Final instruction to coding agent

- Never silently invent product requirements. If a requirement is missing, choose the safest minimal implementation and record the assumption in /docs/ASSUMPTIONS.md.
- Do not add random features because they sound impressive. Build the defined product first.